

1. What does **time complexity** measure?
The number of operations as input size grows.
2. What does **space complexity** measure?
The amount of memory used as input size grows.
3. What does Big-O provide?
An asymptotic **upper bound** on growth rate.
4. In linear search over an array of size **n**, what is the **best-case** complexity?
 $\Omega(1)$ - when we're finding what we're looking for in the first element.
5. In linear search over an array of size **n**, what is the **worst-case** complexity?
 $O(n)$ - when we're finding what we're looking for in the last element.
6. Printing numbers from 1 to n once (one loop) has what complexity?
 $O(n)$ - n operations, one for each array element.
7. Two consecutive loops each running n times has what complexity?
 $O(n)$ - drop constant factor: $O(2n) \rightarrow O(n)$
8. A nested loop where both loops run n times has what complexity?
 $O(n^2)$ - the inner loop runs n times for each of the n outer loops.
9. For $O(n^2 + n)$, what is the simplified (dominant-term) complexity?
 $O(n^2)$ because for $n \rightarrow \infty$, n^2 dominates.
10. On a sorted array, binary search has what time complexity?
 $O(\log n)$
11. What is the best possible time complexity for any comparison-based sorting algorithm?
 $O(n \log n)$: $O(\log_2(n!)) \approx O(n \log_2 n - n) = O(n \log n)$.
12. What is the time complexity of accessing an element by index in a vector of n elements?
 $O(1)$ - it's constant (the same) for all i in $v[i]$.
13. What is the time complexity of inserting an element at the start of a vector?
 $O(n)$ - $O(1)$ to get the iterator at `.begin()`, but $O(n)$ to shift all existing elements one index forward.
14. What is the time complexity of removing an element from the middle of a vector?
 $O(n)$ - $O(1)$ to identify the position, but $O(n)$ to shift all subsequent elements one index backward.
15. What is the space complexity of an array storing n integers?
 $O(n)$ - usually, an int is stored in 4 bytes, so that's $O(4n) = O(n)$.